

Giới thiệu về học tăng cường sâu

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

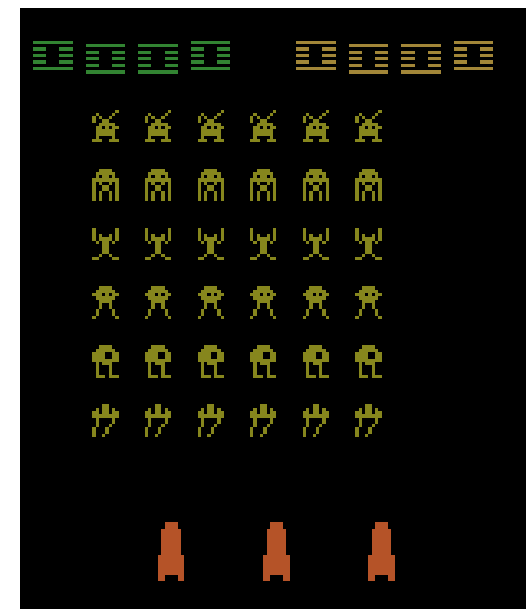


Timothée Carayol

Principal Machine Learning Engineer,
Komment

Vì sao Học tăng cường sâu

- RL truyền thống phù hợp bài toán ít chiều
- Nhiều ứng dụng có không gian trạng thái và/hoặc hành động nhiều chiều



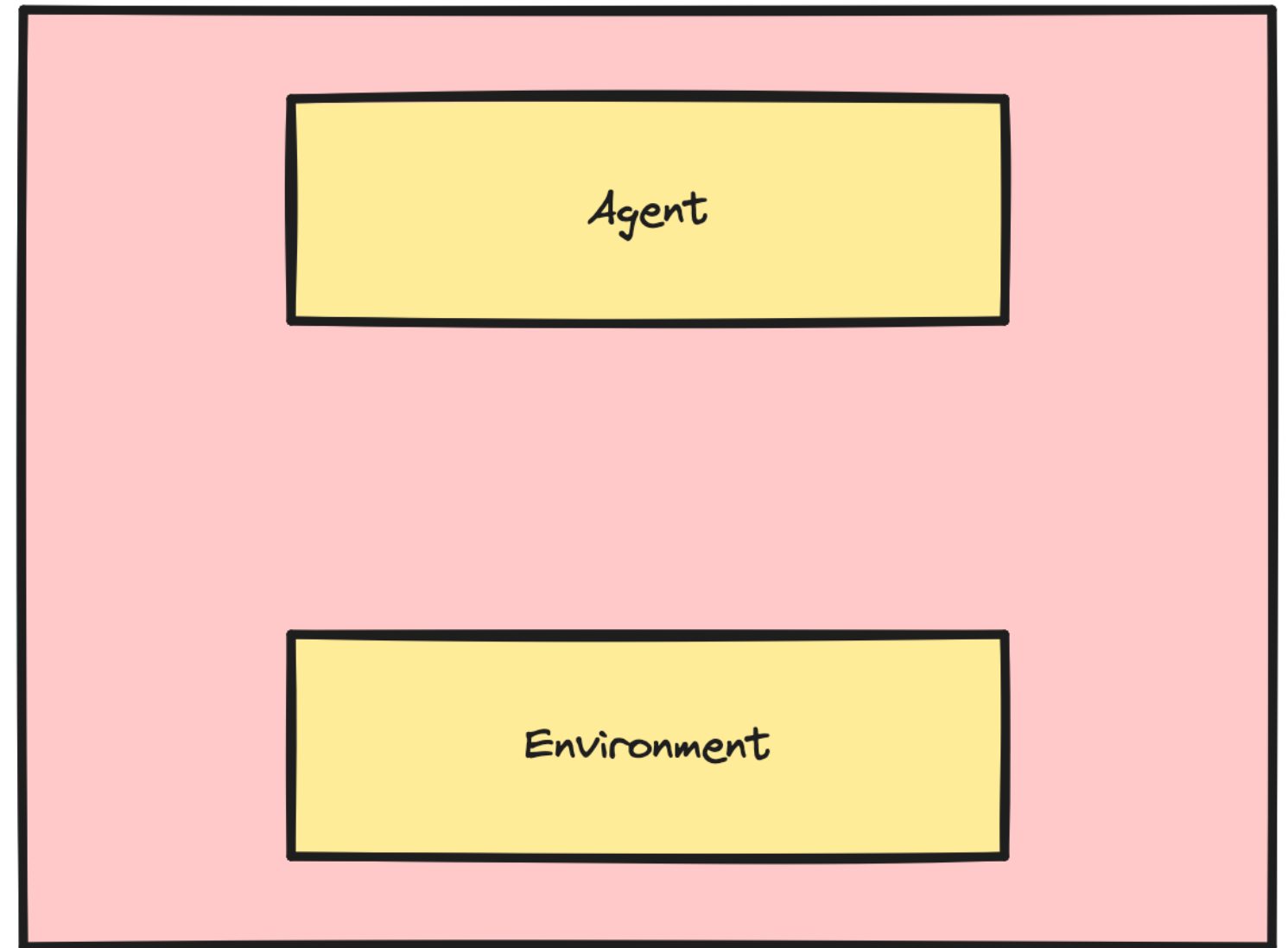
Thành phần của DRL

1. Khái niệm Học tăng cường
 2. Học sâu và PyTorch
- DRL dùng các khái niệm này với mạng nơ-ron sâu



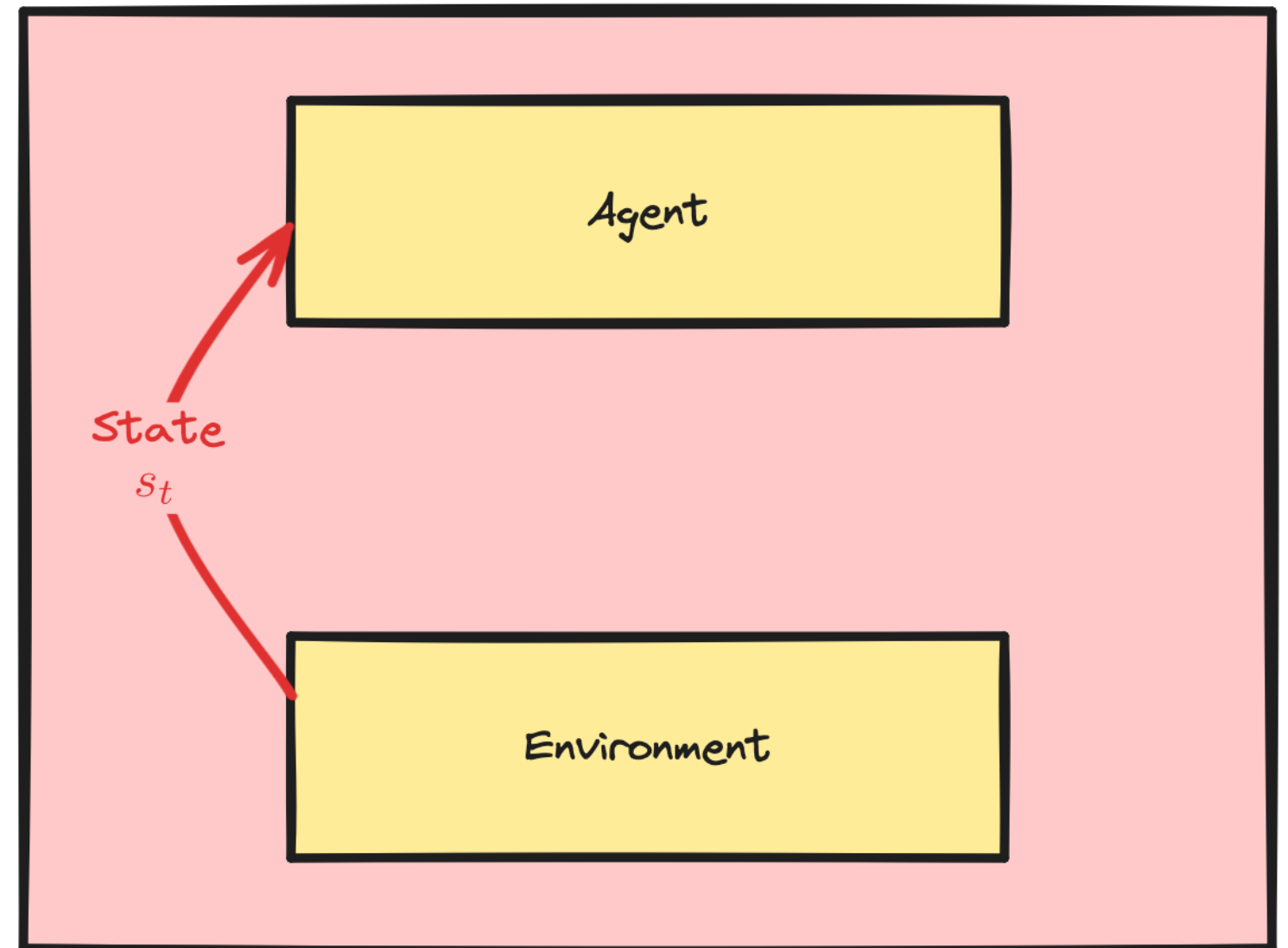
Khung RL

- Bước t:



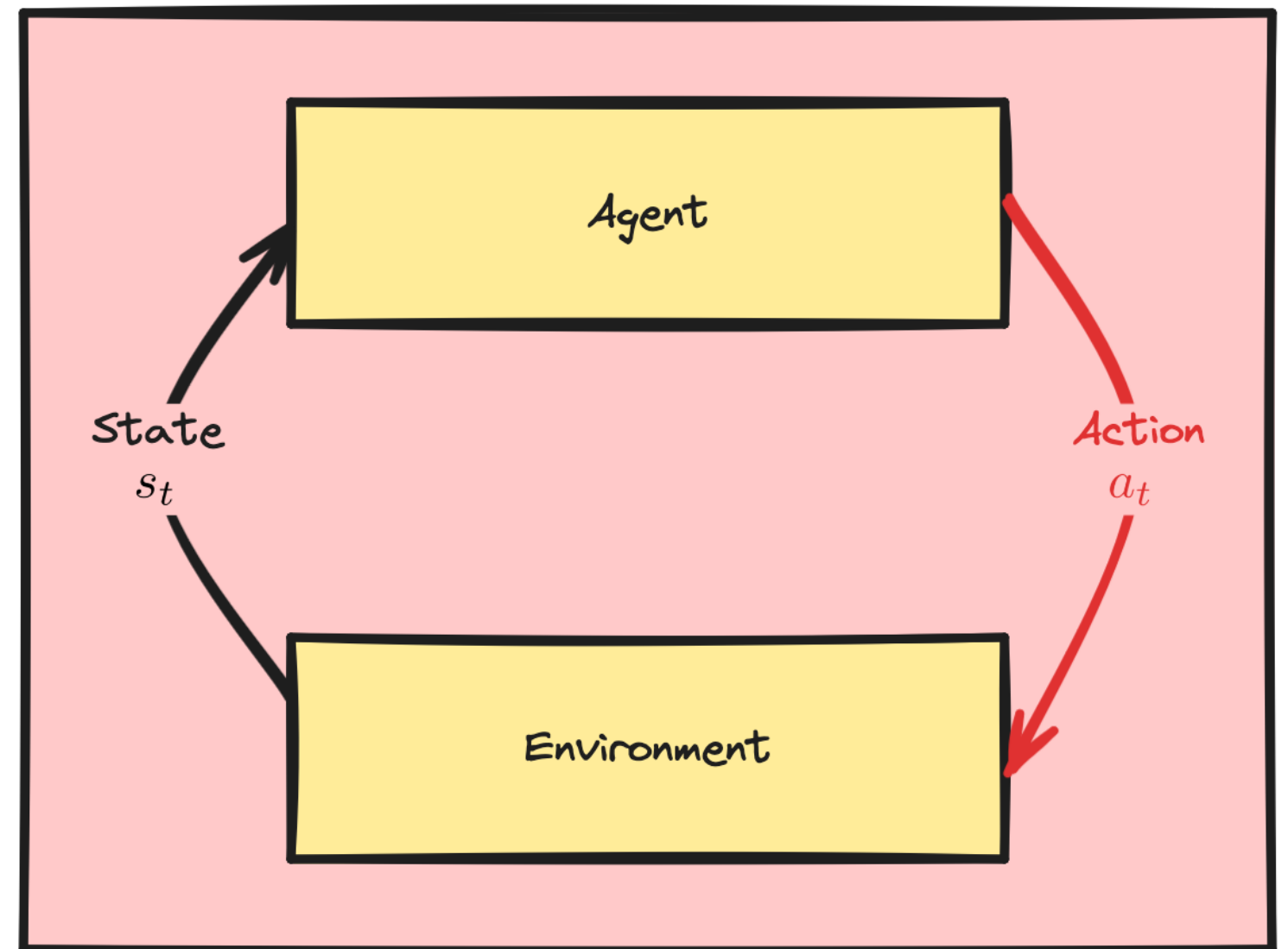
Khung RL

- Bước t:
 - Tác tử quan sát trạng thái s_t



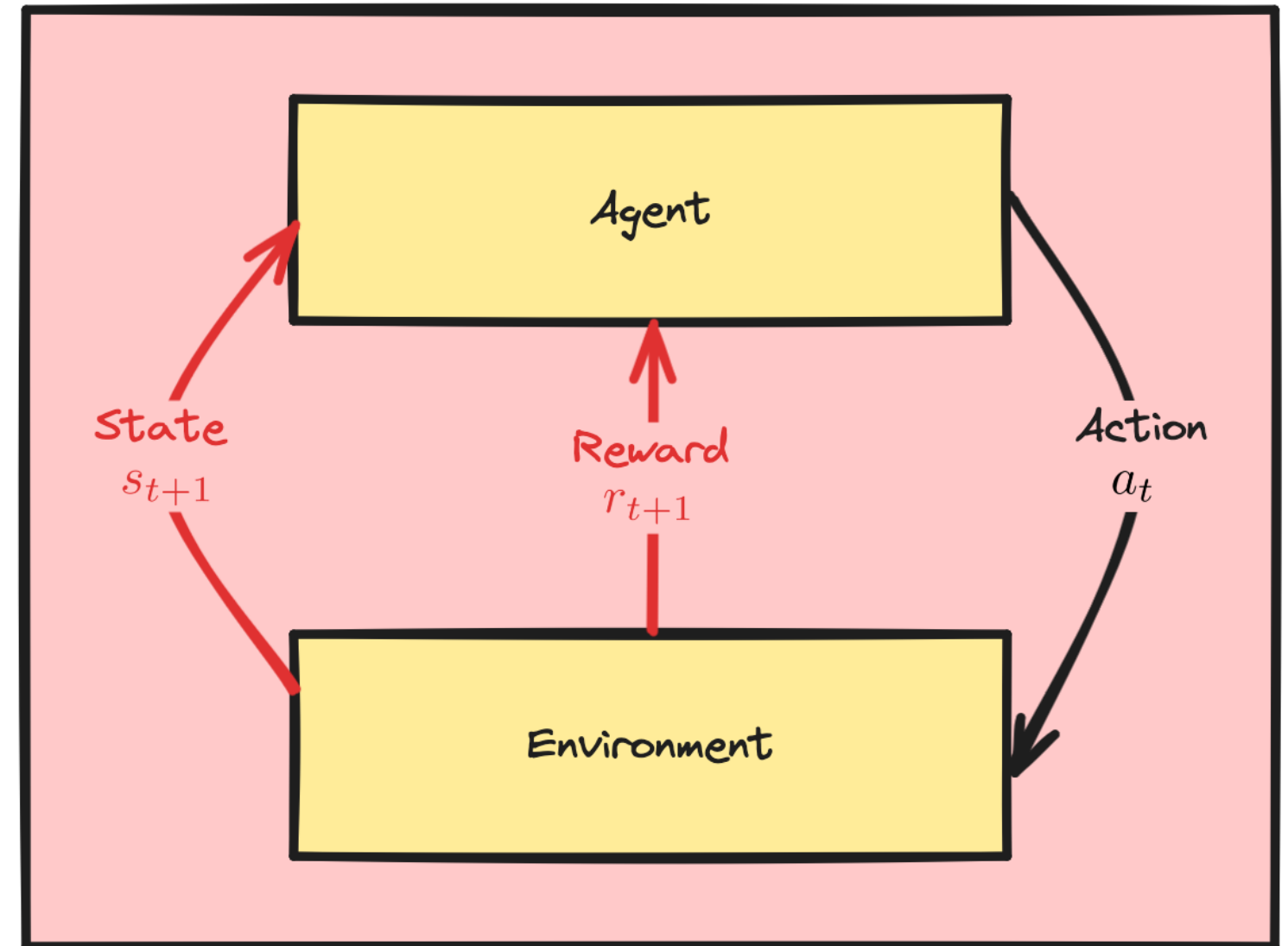
Khung RL

- Bước t:
 - Tác tử quan sát trạng thái s_t
 - Tác tử thực hiện hành động a_t



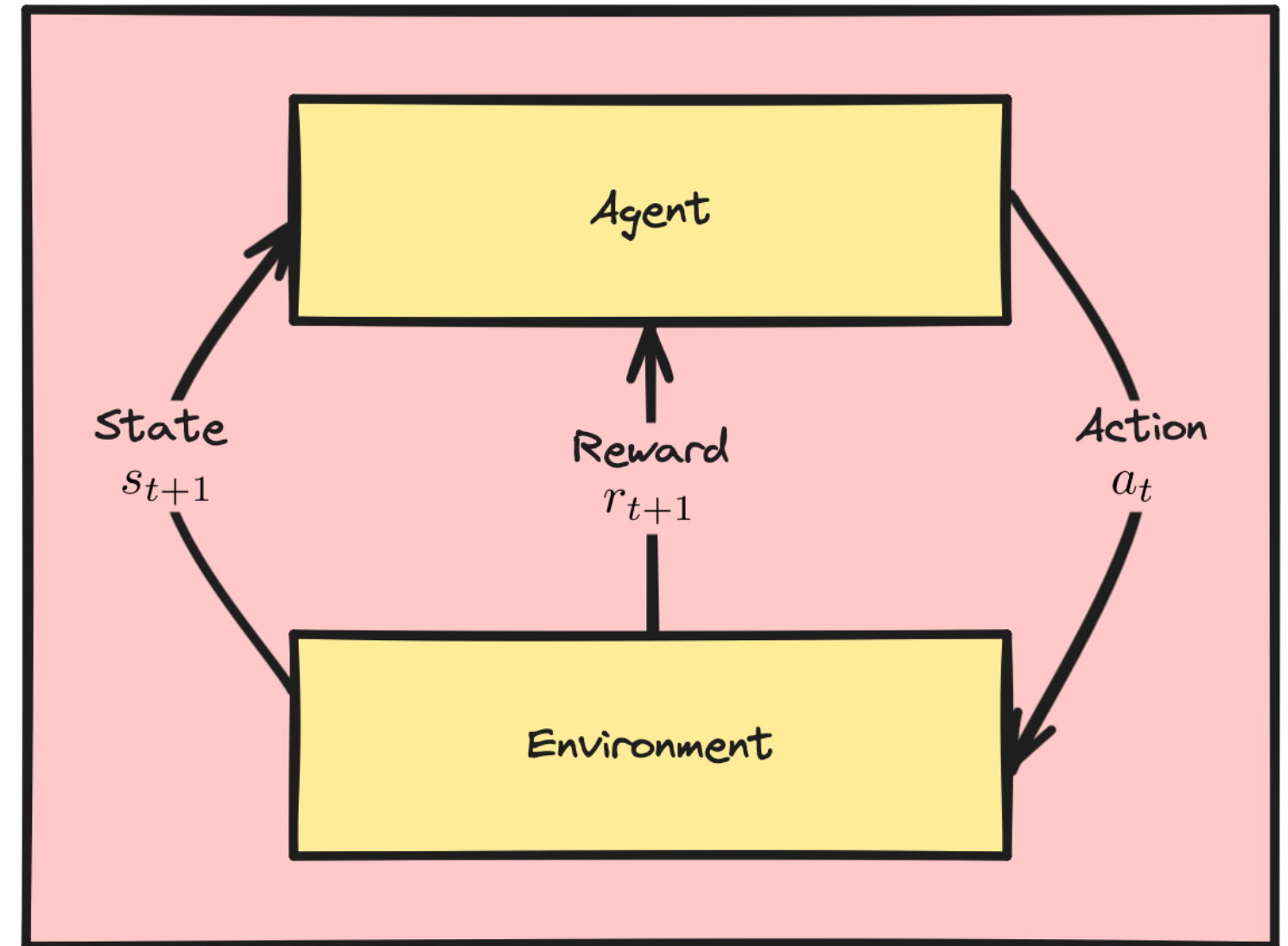
Khung RL

- Bước t:
 - Tác tử quan sát trạng thái s_t
 - Tác tử thực hiện hành động a_t
- Bước t+1:
 - Môi trường trả thưởng r_t
 - Trạng thái chuyển thành s_{t+1}



Khung RL

- Bước t:
 - Tác tử quan sát trạng thái s_t
 - Tác tử thực hiện hành động a_t
- Bước t+1:
 - Môi trường trả thưởng r_t
 - Trạng thái chuyển thành s_{t+1}
- Lặp đến khi kết thúc tập



Chính sách $\pi(s_t)$

- Ánh xạ từ trạng thái tới hành động, mô tả cách tác tử hành xử ở trạng thái s_t
- **Quyết định:**
 - Trả về hành động chọn
- **Ngẫu nhiên:**
 - Trả về phân phối trên các hành động
 - Chính sách là phân phối xác suất trên các hành động có thể

Quỹ đạo và lợi tức tập

Trajectory τ

Sequence of all states and actions in an episode

$$\tau = \left((s_0, a_0), (s_1, a_1), \dots, (s_T, a_T) \right)$$

Episode return R_τ

Total (discounted) rewards accumulated along trajectory τ .

The diagram shows the formula $R_\tau = \sum_{t=0}^T \gamma^t r_t$ with three annotations: an orange arrow labeled 'Episode return' points to R_τ , a purple arrow labeled 'Reward at time t ' points to r_t , and a green arrow labeled 'Discount factor' points to γ^t .

$$R_\tau = \sum_{t=0}^T \gamma^t r_t$$

Thiết lập môi trường

```
env = gym.make("ALE/SpaceInvaders-v5")
# Define neural network architecture
class Network(nn.Module):
    def __init__(self, dim_inputs, dim_outputs):
        super(Network, self).__init__()
        self.linear = nn.Linear(dim_inputs, dim_outputs)
    def forward(self, x):
        return self.linear(x)
# Instantiate network
network = Network(dim_inputs, dim_outputs)
# Instantiate optimizer
optimizer = optim.Adam(network.parameters(), lr=0.0001)
```

Vòng lặp cơ bản

```
for episode in range(1000):
    state, info = env.reset()
    done = False
    while not done:
        action = select_action(network, state)
        next_state, reward, terminated, truncated, _ = (
            env.step(action))

        done = terminated or truncated

        loss = calculate_loss(network, state, action,
                               next_state, reward, done)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    state = next_state
```

- Vòng ngoài: lặp qua các tập
- Vòng trong: lặp qua các bước
 - Chọn hành động
 - Quan sát trạng thái mới và thưởng
 - Tính loss và cập nhật mạng
 - Cập nhật trạng thái
- (Loss?)

Tiếp theo

- DRL rất mạnh!
- Cách tiếp cận dựa trên giá trị và dựa trên chính sách
- DQN và các cải tiến
- Phương pháp gradient chính sách



Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

Giới thiệu về deep Q learning

DEEP REINFORCEMENT LEARNING BẰNG PYTHON



Timothée Carayol

Principal Machine Learning Engineer,
Komment

Deep Q Learning là gì?



Ôn tập Q-Learning

Action Value function $Q_\pi(s, a)$

Sum of future rewards if action a is taken in state s , assuming that the policy π is followed afterwards.

The diagram shows the equation $Q_\pi(s, a) = E_{\tau \sim \pi} [R_\tau | s_t = s, a_t = a]$ with three annotations: a red arrow points from 'Action Value function' to $Q_\pi(s, a)$; a green arrow points from 'Sum of future rewards' to R_τ ; and a blue arrow points from 'Expected value over future trajectories given policy π is followed' to $E_{\tau \sim \pi}$.

$$Q_\pi(s, a) = E_{\tau \sim \pi} [R_\tau | s_t = s, a_t = a]$$

- Biết Q cho phép có chính sách tối ưu:
$$\pi(s_t) = \arg \max_a Q(s_t, a)$$
- Mục tiêu Q-learning: học Q theo thời gian

Ôn tập Q-Learning

Bellman Equation (in Q-learning)

In a deterministic environment:

$$Q_{\pi}(s_t, a_t) = r_{t+1} + \gamma \max_{a_{t+1}} Q_{\pi}(s_{t+1}, a_{t+1})$$

Diagram labels:

- Action Value function (points to $Q_{\pi}(s_t, a_t)$)
- Next reward (points to r_{t+1})
- Discount rate (points to γ)
- Q-value of the best next action (points to $\max_{a_{t+1}} Q_{\pi}(s_{t+1}, a_{t+1})$)

Temporal Difference target

(A.k.a. TD-target, Q-target, or target Q-value.)
Refers to the right side of the Bellman equation, used as target value for the Q-learning update rule.

$$r_{t+1} + \gamma \max_{a_{t+1}} Q_{\pi}(s_{t+1}, a_{t+1})$$

- Phương trình Bellman: công thức đệ quy cho Q
- Vế phải của Bellman: "TD-target"
- Dùng TD-target từ Bellman để cập nhật $\hat{Q}$ sau mỗi bước

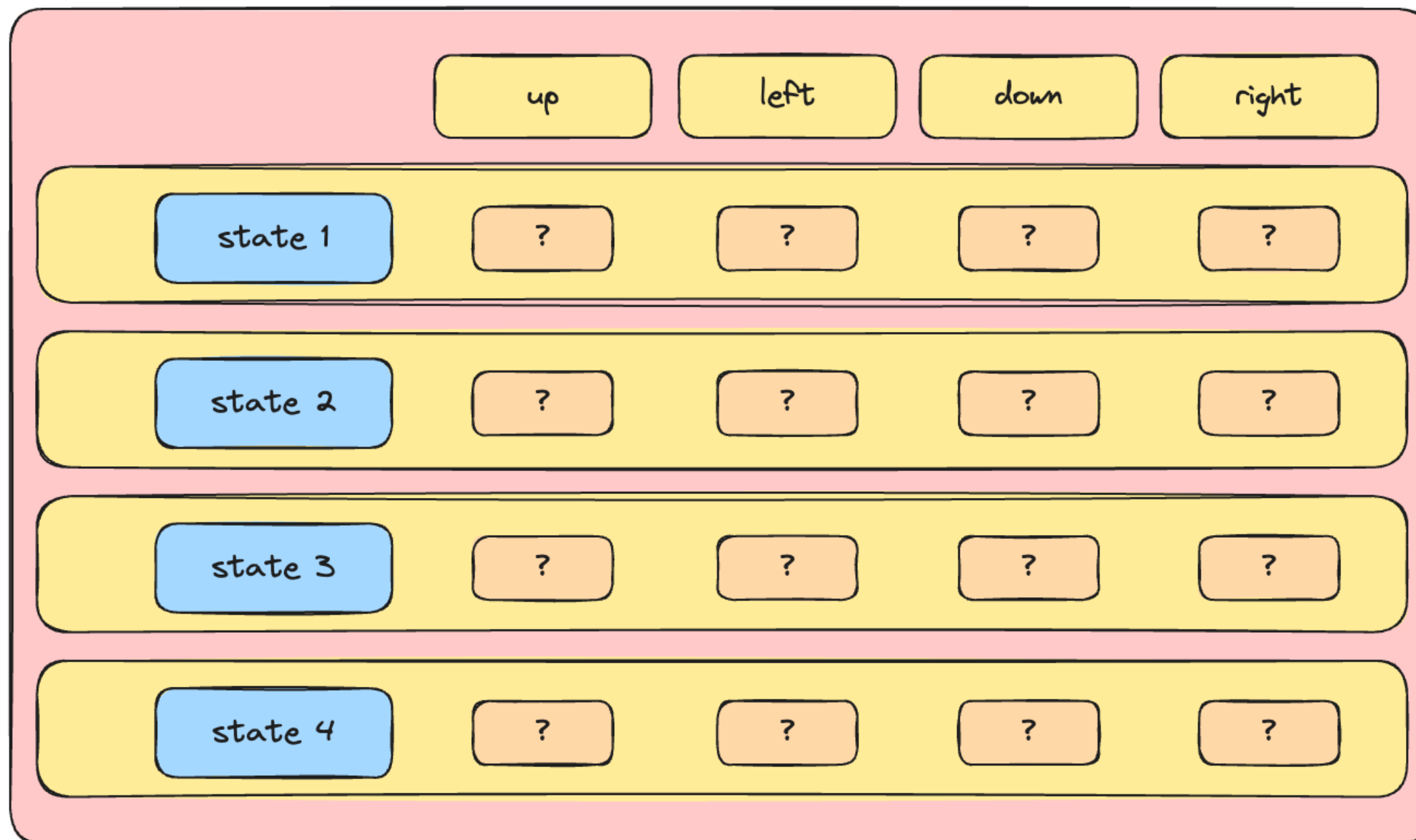
Q-learning update rule

$$\hat{Q}_{new}(s_t, a_t) = (1 - \alpha) \hat{Q}_{old}(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}_{old}(s_{t+1}, a_{t+1}) \right)$$

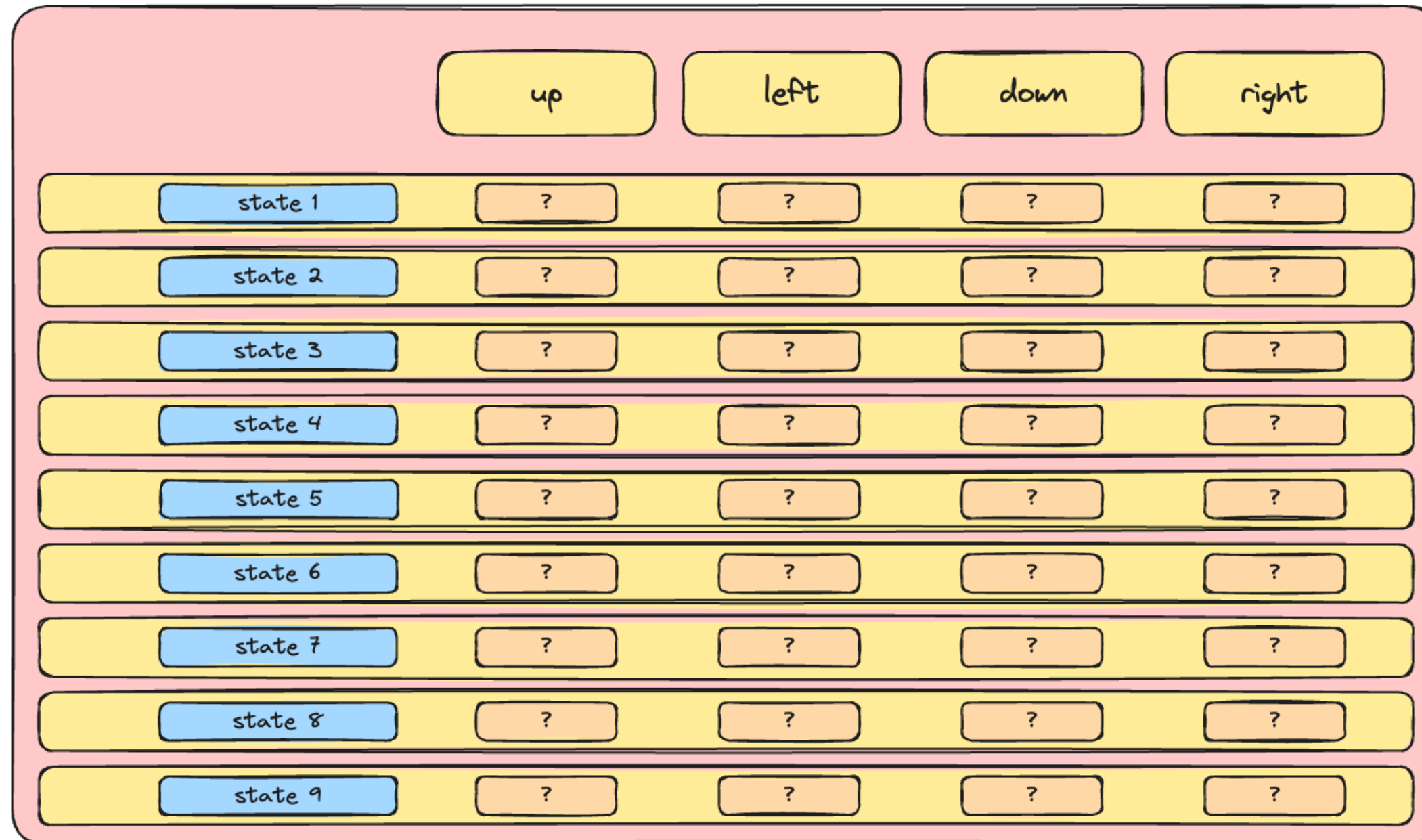
Diagram labels:

- Updated Q-value (points to $\hat{Q}_{new}(s_t, a_t)$)
- Learning rate α (points to α)
- Previous Q-value (points to $\hat{Q}_{old}(s_t, a_t)$)
- TD-target (points to the term in parentheses)

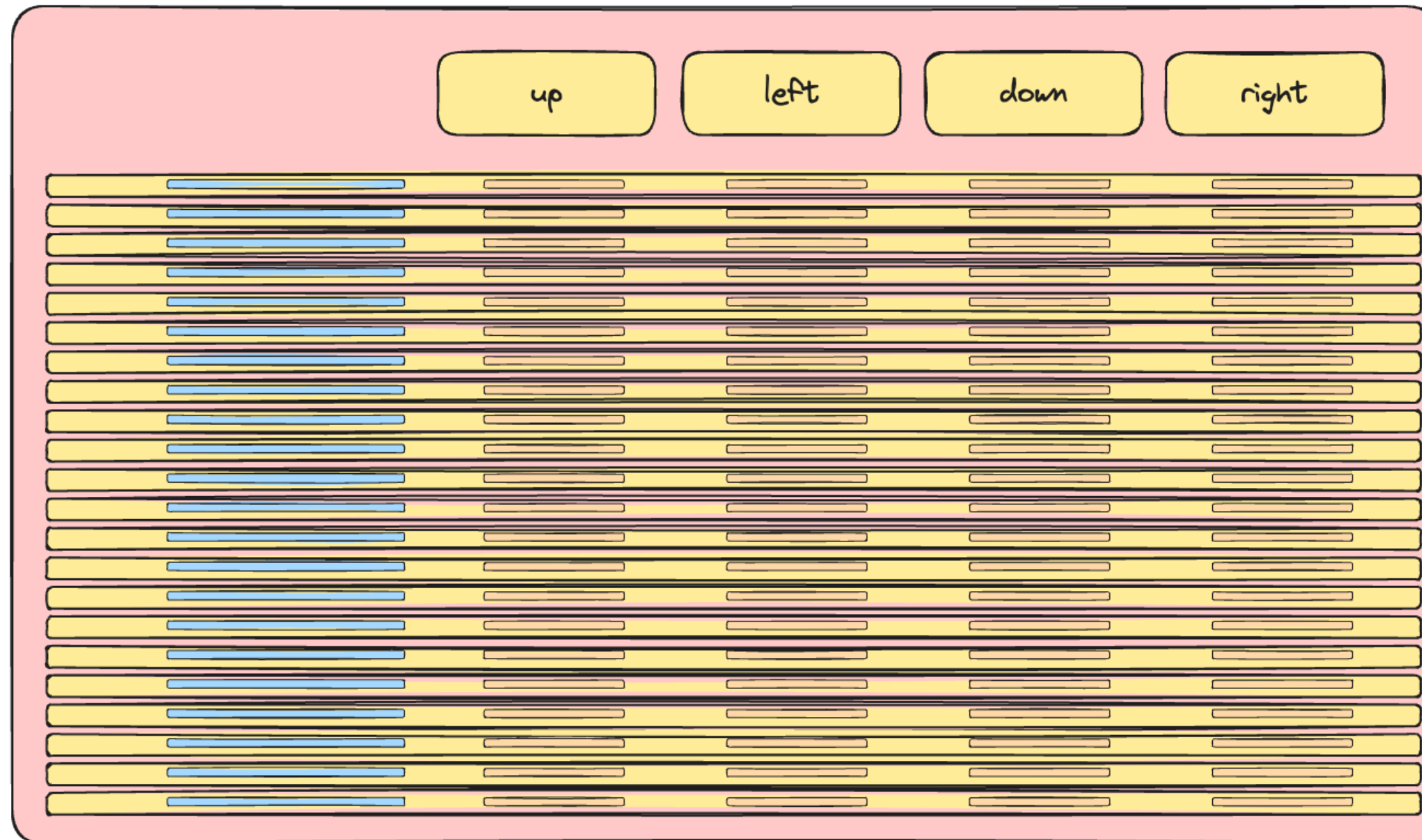
Q-Network



Q-Network

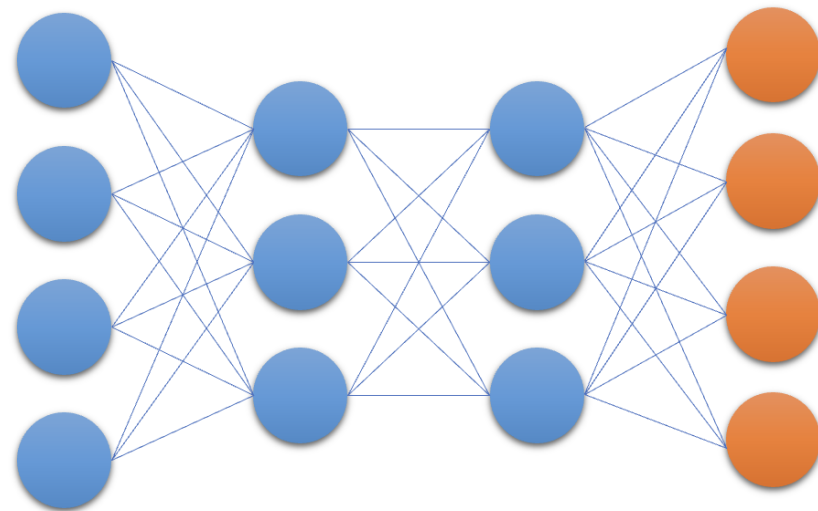


Q-Network



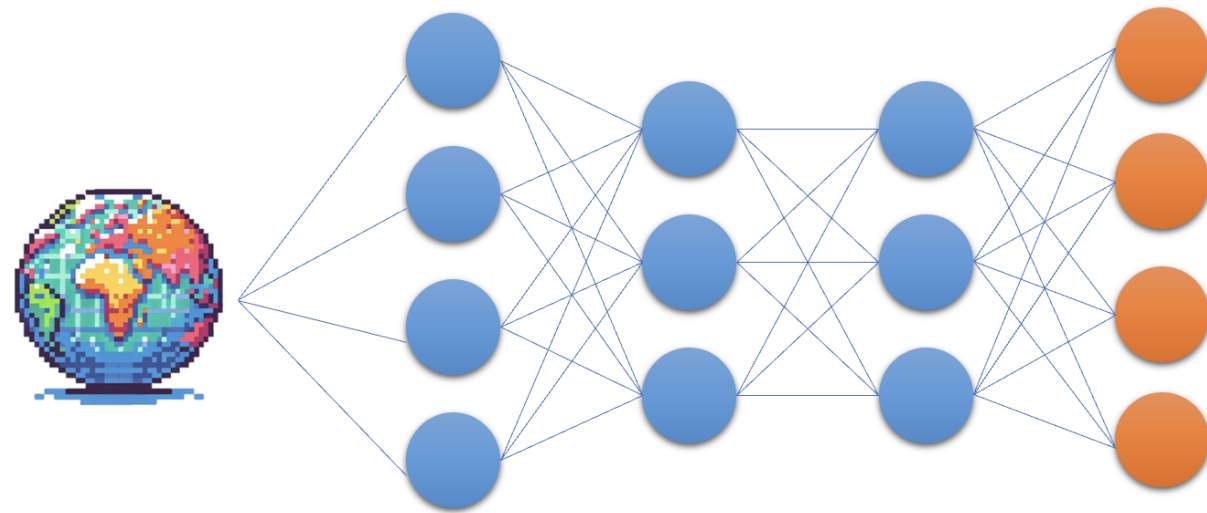
Q-Network

- Trọng tâm của Deep Q Learning: một mạng nơ-ron



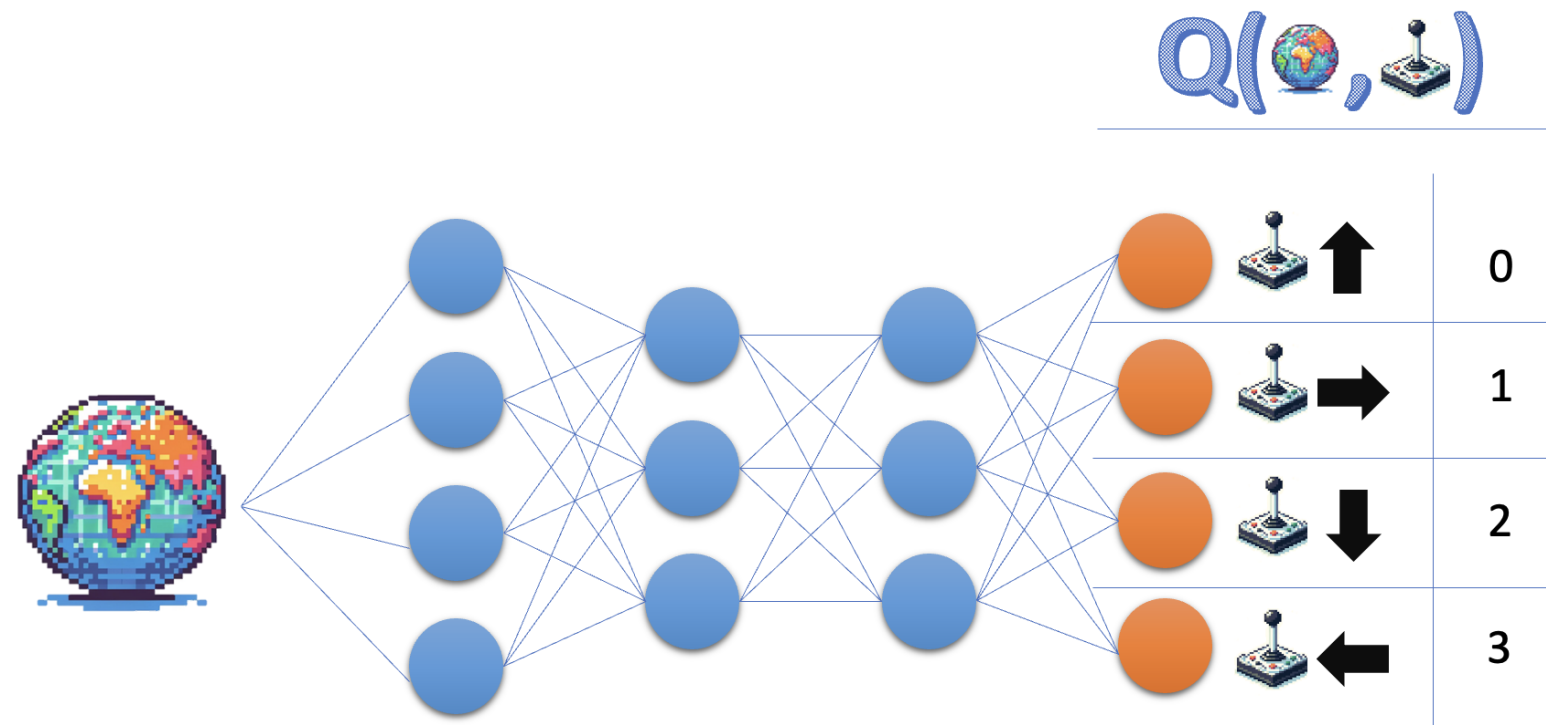
Q-Network

- Trọng tâm của Deep Q Learning: một mạng nơ-ron



Q-Network

- Trọng tâm của Deep Q Learning: mạng nơ-ron ánh xạ trạng thái → Q-value



- Mạng xấp xỉ hàm giá trị-hành động gọi là "Q-network"
- Q-network thường dùng trong các thuật toán Deep Q Learning như DQN.

Cài đặt Q-network

```
class QNetwork(nn.Module):

    def __init__(self, state_size, action_size):
        super(QNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, state):
        x = torch.relu(self.fc1(torch.tensor(state)))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

q_network = QNetwork(8, 4)
optimizer = optim.Adam(q_network.parameters(),
                        lr=0.0001)
```

- Kích thước đầu vào do trạng thái quyết định
- Kích thước đầu ra do số hành động quyết định
- Ví dụ này:
 - 2 tầng ẩn, mỗi tầng 64 nút
 - Kích hoạt ReLU

Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

Thuật toán DQN tối giản

DEEP REINFORCEMENT LEARNING BẰNG PYTHON



Timothée Carayol

Principal Machine Learning Engineer,
Komment

DQN tối giản

- Bước đầu tới thuật toán DQN đầy đủ
- Thành phần:
 - Vòng lặp huấn luyện DRL tổng quát
 - Mạng Q
 - Nguyên lý Q-learning

```
for episode in range(1000):
    state, info = env.reset()
    done = False
    while not done:
        # Action selection
        action = select_action(network, state)
        next_state, reward, terminated, truncated, _ = (
            env.step(action))

        done = terminated or truncated
        # Loss calculation
        loss = calculate_loss(network, state, action,
                               next_state, reward, done)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        state = next_state
```

Chọn hành động trong DQN tối giản

```
def select_action(q_network, state):  
    # Feed state to network to obtain Q-values  
    q_values = q_network(state)  
    # Obtain index of action with highest Q-value  
    action = torch.argmax(q_values).item()  
    return action
```

```
Q-values: [-0.01, 0.08, 0.12, -0.07]  
Hành động chọn: 2, q-value 0.12
```

- Chính sách: chọn hành động có Q-value cao nhất
 - $a_t = \arg \max_a Q(S_t, a)$
 - Ở đây: hành động 2, Q-value 0.12

Hàm loss DQN tối giản

- Hàm giá trị-hành động thỏa phương trình Bellman
- Ý tưởng: tối thiểu hóa chênh lệch hai vế (TD-error/Bellman error)
- Dùng bình phương Bellman error làm hàm loss:

DQN loss (Squared Bellman Error)

$$\mathcal{L}(\theta) = \left[\left(\underbrace{r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}_{\theta}(s_{t+1}, a_{t+1})}_{\text{TD target}} \right) - \underbrace{\hat{Q}_{\theta}(s_t, a_t)}_{\text{Current Q value estimate}} \right]^2$$

Bellman Equation (in Q-learning)

$$Q_{\pi}(s_t, a_t) = \underbrace{r_{t+1}}_{\text{Next reward}} + \underbrace{\gamma}_{\text{Discount rate}} \underbrace{\max_{a_{t+1}} Q_{\pi}(s_{t+1}, a_{t+1})}_{\text{Q-value of the best next action}}$$

Bellman Error a.k.a. TD-error

$$\left(\underbrace{r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}(s_{t+1}, a_{t+1})}_{\text{TD target}} \right) - \underbrace{\hat{Q}(s_t, a_t)}_{\text{Current Q value estimate}}$$

Hàm loss DQN tối giản

```
def calculate_loss(
    q_network, state, action,
    next_state, reward, done):
    q_values = q_network(state)
    current_state_q_value = q_values[action]
    next_state_q_value = q_network(next_state).max()
    target_q_value = reward +
        gamma * next_state_q_value * (1-done)
    loss = nn.MSELoss()(
        current_state_q_value, target_q_value)
    return loss
```

- Q-value trạng thái hiện tại:

$$Q(s_t, a_t)$$

- Q-value trạng thái kế tiếp:

$$\max_a Q(s_{t+1}, a)$$

- Q-value mục tiêu:

$$r_{t+1} + \gamma \max_a Q(s_{t+1}, a)$$

- Loss của DQN:

$$\left(Q(s_t, a_t) - (r_{t+1} + \gamma \max_a Q(s_{t+1}, a)) \right)^2$$

Mô tả các tập

```
describe_episode(episode, reward, episode_reward, step)
```

Tập	1	Thời lượng:	84 bước	Lợi tức:	-871.38	Rơi vãi	
Tập	2	Thời lượng:	53 bước	Lợi tức:	-452.68	Rơi vãi	
Tập	3	Thời lượng:	57 bước	Lợi tức:	-414.22	Rơi vãi	
Tập	4	Thời lượng:	54 bước	Lợi tức:	-475.09	Rơi vãi	
Tập	5	Thời lượng:	67 bước	Lợi tức:	-532.31	Rơi vãi	
Tập	6	Thời lượng:	53 bước	Lợi tức:	-407.00	Rơi vãi	
Tập	7	Thời lượng:	52 bước	Lợi tức:	-380.45	Rơi vãi	
Tập	8	Thời lượng:	55 bước	Lợi tức:	-380.75	Rơi vãi	
Tập	9	Thời lượng:	88 bước	Lợi tức:	-688.68	Rơi vãi	
Tập	10	Thời lượng:	76 bước	Lợi tức:	-338.06	Rơi vãi	

Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON